

Design Strategy for a Generative E-Ink Visualization Engine

Overview and Goals

This project envisions an **ambient, generative art display** driven by smart home data. Unlike a traditional dashboard, the goal is to produce a **“living” grayscale artwork** on a Waveshare 16-level e-ink panel, reflecting home activity in subtle, poetic ways. The system must consolidate hundreds of Home Assistant (HA) sensor entities into a few **semantic input channels** (e.g. *house activity*, *soundscape*, *resource use*, *climate comfort*) that drive the visuals. It should be **modular and extensible** – new devices or visual mappings can be added via configuration rather than code – and optimized for e-ink’s unique performance characteristics (partial refresh, low refresh rate, ghosting, dithering, etc.). We will outline a comprehensive architecture, from data ingestion to rendering, and discuss best practices (including known community tricks) to balance hardware limitations with artistic richness.

Mapping Home Data to Semantic Channels

Instead of mapping each sensor directly to a visual element (which would create an overload of information), the engine uses **normalized “proxy” measures** that aggregate and abstract the raw data. This follows the strategy of systems like *Tableau Machine*, which achieved an enchanting ambient display by *“mapping computed proxy measures to the design attributes of the 2D composition, rather than to individual [sensor] elements”*, thereby encouraging interpretation without one-to-one literalness ¹. Direct, literal mappings (like showing every sensor reading) feel like surveillance or rote regurgitation, while completely random visuals are unintelligible; a **midpoint** of data-driven but **ambiguous representation** sustains long-term interest ² ³. In practice, this means defining a handful of **semantic channels** that each combine multiple related sensors into a single numeric or categorical value which influences the artwork.

Proposed Channels and Inputs:

- **House Activity:** A composite measure of occupancy and motion. It could synthesize *motion sensors* (Hue, Aqara PIRs), *entry sensors* (doors/locks, garage door openers), and presence (e.g. Eero router device presence or phone trackers) into an “activity level” metric. For example, each motion event or door unlock might spike this channel’s value (with a decay over time), producing a higher “energy” when the home is busy. Even aggregate stats like number of rooms active or frequency of recent events can feed this channel.
- **Soundscape:** Represents ambient sound or music activity. This might tap into **Apple HomePod** status (e.g. whether music is playing and at what volume) or any sound level sensors available. A HomePod integrated as a `media_player` in HA can indicate if audio is playing; multiple active speakers or higher volume could increase the *soundscape* intensity. (If actual dB noise sensors were present, they would contribute as well.)
- **Resource Use:** Abstracts the consumption of resources like water, energy or network. For instance, the **Flume water monitor** reporting current water flow or daily usage can map to this channel – high water flow might raise a “water usage” sub-component. If energy monitoring is available (not listed, but possibly via smart plugs or the power draw of devices on Eero), that could factor in too. In absence of direct power sensors, we might use proxies like the number of devices online (from Eero) or HVAC usage from the

thermostat as a rough indicator of energy use. The *resource use* channel could thus combine water usage rate, any available power metrics, or internet bandwidth usage into a single scale of resource consumption.

- **Climate Comfort:** An indicator of environmental comfort inside the home. This would draw on **Ecobee thermostat** data (e.g. difference between current temperature and target setpoint, HVAC running state) and environmental sensors (Aqara or Hue devices often provide temperature/humidity). For example, if several rooms report temperatures far from their setpoints or high humidity, the comfort level might drop. Blinds (IKEA smart blinds) position can also influence this – open blinds on a hot day might decrease comfort (or increase sunlight exposure). The channel might be normalized such that 0% = very uncomfortable (too hot/cold/humid) and 100% = optimal comfort.

Each channel's value is computed by a **normalization module** that translates diverse sensor units into a unified 0–1 or low–high scale. Configuration files (e.g. YAML/JSON) will define these mappings. For example, a YAML config might list each relevant HA entity and how it affects a channel:

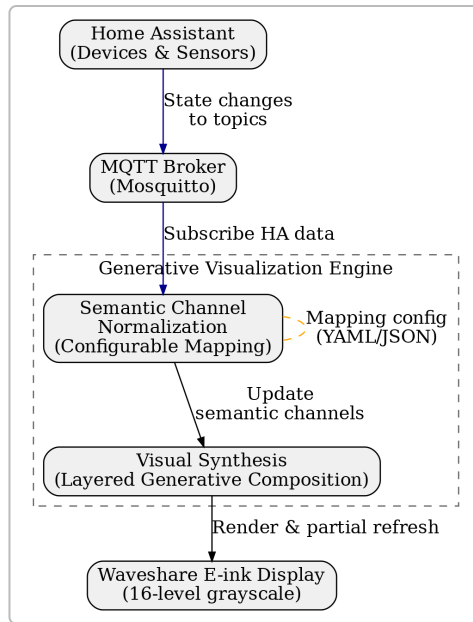
- *house_activity*: include `binary_sensor.living_room_motion` (weight 1.0 on motion event), `sensor.front_door_lock` (trigger burst when unlock), etc., with parameters like decay rates for activity.
- *soundscape*: link `media_player.kitchen_homepod.volume` (map 0–1 volume to channel) and `media_player.living_room_homepod.state` (add fixed value if playing).
- *resource_use*: link `sensor.flume_water_gpm` (map 0–10 GPM flow to 0–1 scale) and perhaps `sensor.internet_bandwidth` (normalized) or `sensor.thermostat_hvac_power` if available.
- *climate_comfort*: link `sensor.living_room_temperature` (with a target comfort range) and `sensor.living_room_humidity`, as well as `climate.home_thermostat.hvac_action` (if heating/cooling is running, comfort is not ideal).

This config-driven approach makes the system **extensible** – adding a new device simply means updating the config to include it in the appropriate channel. Internally, the normalization logic can be generic, applying simple formulae (like linear scaling, thresholds, or event triggers) as specified in the config.

Notably, these channels are *designed to be semantically meaningful but not overly explicit*. The display will **not show raw sensor values** or icons for each device; instead it will use these channels to influence an abstract visual. This ensures the output remains *artistic and ambient*. Home inhabitants should be able to sense patterns (e.g. the display grows more animated when many people are active, or a gentle visual quiet when the house is still at night) without reading numbers. Prior research in ambient displays shows that such abstract yet responsive visuals can increase awareness while “*not exposing specific values*” or demanding focused attention ⁴ ⁵ .

System Architecture and Data Pipeline

The system is composed of three main parts: **data ingestion from Home Assistant**, the **generative visualization engine**, and the **e-ink display driver**. The diagram below shows the high-level architecture and data flow.



System architecture for the generative e-ink visualization engine. Home Assistant pushes device state changes to an MQTT broker; the visualization engine subscribes and updates semantic channels (via configurable mappings), then generates layered graphics that are rendered to the 16-level grayscale e-ink display. This modular design allows new devices or visual mappings to be added via config without changing core code.

Home Assistant Integration (Data Ingestion)

To feed real-time data into the visualization engine, we leverage Home Assistant’s integration capabilities. Two reliable patterns can be used here:

- **MQTT Event Stream:** Home Assistant’s built-in **MQTT integration** (specifically the *MQTT Statestream*) can publish state changes to an MQTT broker on your network. This integration can be configured to “publish state changes in Home Assistant to individual MQTT topics” for selected entities ⁶. For example, when a motion sensor changes state or a new temperature reading is available, HA will publish a message to a topic like `home/<entity_id>/state`. The visualization engine acts as an MQTT client (using a library such as `paho-mqtt` in Python or an MQTT addon in C++) subscribing to the relevant topics. This decouples the system nicely: Home Assistant just announces updates, and the engine reacts to them asynchronously. The MQTT broker (e.g. Mosquitto) could run on the Home Assistant host or separately. Using MQTT is advantageous for a modular system, since adding a new device just means subscribing to one more topic, and even remote or low-power devices (ESPHome sensors, etc.) already speak MQTT.
- **WebSocket API:** Alternatively, the engine can interface with Home Assistant’s **WebSocket API**, which “can be used to stream information from a Home Assistant instance to any client” ⁷. The engine would connect to HA’s WebSocket endpoint (authenticated via a long-lived access token) and subscribe to event types or `state_changed` events for entities of interest. This provides real-time push updates without an external broker. There are Python client libraries (and examples using `asyncio` websockets or the `homeassistant` Python package) that make subscribing to HA events

straightforward. The WebSocket approach avoids needing MQTT, but is slightly more tightly coupled to Home Assistant.

Both methods are valid; in many setups MQTT is already used (for device communication), so the Statestream approach piggybacks on that. We will assume an **MQTT-based pipeline** for clarity. Home Assistant publishes updates, and the **engine's data ingestion component listens and updates the semantic channel values** accordingly.

Config-driven Mapping: The first component inside our engine is the **Semantic Channel Normalization** module (see diagram) which processes incoming events. This module uses the user-defined config mappings to decide how each HA entity's data affects the channels. For instance, if a message comes in for `sensor.kitchen_motion` = `on`, the module sees from config that this adds, say, +0.3 to the `house_activity` channel (which then decays gradually). A thermostat temperature update might be compared against the comfort range and update the `climate_comfort` metric. The normalization module maintains the current value/state of each semantic channel in memory. It may also apply smoothing or rate limiting – e.g. averaging readings over a short window, or ensuring abrupt spikes are toned down – to prevent the visualization from jittering with every tiny sensor fluctuation. Each channel value is thus a continuously updated, normalized signal.

This design makes it easy to **add or remove data sources**: the config can include an *include list* of entity IDs for each channel (or conversely an *exclude* for irrelevant ones). Because the system handles many entities, it's wise to filter to only those needed – the MQTT Statestream can be set up to publish only selected entities to reduce noise ⁸. (Alternatively, one could run a lightweight filter in the engine to ignore topics not in config.)

Visualization Engine Core

The heart of the system is the **Generative Visualization Engine**. It takes the semantic channel values and produces an **image (frame) for the e-ink display**. The engine is designed in a **layered, modular fashion** to maximize creativity and extensibility:

- **Layered Visual Synthesis:** Each semantic channel (or combination of channels) controls one aspect or “layer” of the visual composition. Instead of drawing one monolithic image in one go, the engine can maintain multiple layers (e.g. backgrounds, foreground elements, overlays) that correspond to different data-driven effects. For example, a *background layer* might represent climate comfort as a slowly morphing texture or gradient: if the house is comfortable, it could render a calm, uniform pattern; if discomfort rises (too hot/cold), perhaps a noisy or fractal pattern subtly appears, like visual “heatwaves” or “chill” noise. A *motion/activity layer* could consist of particles or abstract shapes that are generated when motion events occur – e.g. a dot or ripple emanating from a position that might correspond to a room, or simply an increase in the number of floating shapes on the canvas when activity is high. A *sound layer* might visualize music playing as gentle pulsating rings or a waveform stripe that animates with the beat or volume (since e-ink is slow, this would not be real-time audio waveform, but a slowly updating pattern, perhaps a bar that grows with volume or circles that expand when sound is on). A *resource use layer* could introduce geometric patterns (like water ripple concentric circles or pixelated clouds) whose density or size correlates with water flow or energy use. All layers are then composited to form the final frame. Composition could be simply drawing them on top of each other with appropriate transparency or blending (in 16 shades of gray)

– care should be taken that the combined result still respects the e-ink’s display limits (since it’s grayscale, layering might just mean different gray intensities or dither patterns overlapping).

- **Generative Algorithms:** Emphasis is on **procedural generation** rather than fixed images. Techniques from creative coding can be employed: noise functions (Perlin/Simplex noise) to generate organic textures that can be modulated by sensor values, particle systems whose behavior rules (speed, count, spawn rate) are driven by channel intensities, L-systems or abstract drawings that evolve based on inputs, etc. The engine should allow plugging in new algorithms easily. For modularity, one can structure the code so that each channel or layer is handled by a separate class or module (e.g. a “ParticleLayer” for activity, a “NoiseBackgroundLayer” for climate, etc.), each with a function to update/draw itself given the current channel value. Adding a new mapping or visual effect then means adding a new layer module and referencing it in a config.
- **Expressiveness over Precision:** Because this is an art display, the visual mapping can be metaphorical. For instance, *house activity* might correspond to the **movement** of elements (lots of motion = faster or more chaotic movement on screen), while *soundscape* might correspond to a **rhythmic pulsing** or pattern change (music playing triggers a faint oscillating border or graphical “equalizer” bar in the composition). *Resource use* could alter the **brightness or texture** – e.g. heavy resource consumption might add a granular texture or darker tint to the image (a subtle cue analogous to “stress” on the environment). These choices are artistic; the key is that over time users can intuitively correlate them (e.g. “*The background turns speckled when we’re using a lot of water or power*” or “*When the house is busy, these shapes start dancing around*”). User studies of ambient art have found that people will *interpret the display in terms of their activities*, even if the system isn’t explicitly literal – which is exactly the goal ⁹ ¹⁰ . For example, they might say “the display is calm because we’re quiet” or joke that the system gets “unstable” if one group of occupants doesn’t do chores ¹¹ ¹² , indicating the display successfully prompts reflection without showing any direct text or alerts.
- **Aesthetic Cohesion:** All layers should be designed to form a cohesive visual style (e.g. a minimalistic grayscale palette, possibly inspired by nature or abstract art). Since the display is 16-level grayscale, we can use subtle shading and patterns. The design should **avoid a cluttered look**; negative (white) space is valuable on e-ink to maintain a calm appearance. It may help to impose an overall visual theme (for instance, imagining the display as a kind of generative “landscape” of the home’s state – calm skies vs. noisy clouds, etc., metaphorically). The architecture supports this by allowing a custom rendering function for each layer that can incorporate any logic.

From a **technical implementation** standpoint, there are a few options to develop this engine:

- **Python-based Approach:** Python is convenient for tying into Home Assistant (via MQTT or API) and for reading config files. Libraries like **Pillow (PIL)** or **Pycairo** can be used to draw shapes, text, or images onto an in-memory canvas (an image buffer). Pillow supports basic drawing (points, lines, text) and composition of images; this could suffice for simple generative graphics or for rendering precomputed patterns. However, complex real-time generative art (many particles, fluid noise) in Python might be slow, especially on a Raspberry Pi. Python can still drive the high-level logic and possibly call out to faster libraries (NumPy for pixel operations, or even OpenCV for certain image filters). One approach is to prototype visuals in Python (for ease of development) and optimize hotspots with libraries or by moving to C/C++ if needed.

- **openFrameworks (C++):** The prompt suggests using openFrameworks, which is a C++ creative coding framework known for real-time graphics. openFrameworks can run on embedded devices (including Raspberry Pi) and on macOS, making it a good choice for cross-platform deployment. We can build the visualization engine as an openFrameworks app that subscribes to MQTT (there are addons for MQTT or one can use C++ MQTT client libraries) and then use openFrameworks' drawing API to render to an image or directly to the screen. The **advantage** of openFrameworks is performance – it can handle more sprites, higher resolution noise, etc., at the cost of more complex development. Since e-ink is not a high frame-rate display, we don't need video-like 60fps rendering, but we might want to smoothly animate something at e.g. 1–2fps or update small parts frequently, which C++ can handle more efficiently. The openFrameworks app could either draw into an offscreen buffer and send that to the e-ink driver, or possibly use a Linux framebuffer driver for the e-ink (if available) so that drawing calls go directly to the e-ink as if it were a window. (On a Raspberry Pi, Waveshare e-ink HATs typically interface via SPI, not as a standard monitor, so we'd likely need to use Waveshare's library or our own SPI code. We can still have openFrameworks generate the pixel data, then call a function to push it to SPI.)
- **Hybrid:** It's possible to use Python for data + config management and offload graphics to a specialized process. For example, the Python side could send high-level drawing commands or parameters to a separate process (maybe even an ESP32 or another microcontroller that drives the e-ink). However, given the desire for rich generative visuals, it's more straightforward that the same application handles both data and drawing. A clean modular approach might be to separate the concerns: a Python service could collect/normalize HA data and publish a small set of *"visual state" messages (via MQTT or shared memory)* that a visualization program (in openFrameworks or Processing, etc.) subscribes to. This way, the visuals code doesn't need to have HA integration built-in; it only needs to receive the already-processed channel values. This decoupling means the visuals module could be swapped out or run on a different machine. For instance, the **Mac mini (M4)** could run the heavy visualization and then send frames to the Raspberry Pi attached to the e-ink. In fact, one proven pattern (used in some DIY e-ink art projects) is to generate images on a powerful machine and send them to a lightweight display driver: *"We built an e-ink frame that shows AI-generated art by generating images on a desktop and serving them to ESP32/ Raspberry Pi frames via a local server"* ¹³ ¹⁴ . In our case, the Mac mini could run the generative art software (taking advantage of its GPU for any rendering, if needed), and the Raspberry Pi could be a thin client that simply receives a new image (e.g. via MQTT's binary payload or an HTTP request) and writes it to the e-ink. This is probably overkill for simpler graphics, but it's an option if the visuals become very computational (or if one wants to leverage Mac-specific tools like Core Graphics or Metal for rendering). On the other hand, the **Raspberry Pi** (especially a Pi 4 or Pi 5) is capable enough to run moderately complex openFrameworks sketches on its own, so it may be simpler to run everything on the Pi for a self-contained device attached to the e-ink panel.

Development Stack Suggestions:

- Use **Python** for the integration/config layer (parsing YAML, connecting to MQTT/WebSocket, managing channel state). Python's rich ecosystem (e.g. `homeassistant` API, `paho-mqtt`, `NumPy`) makes these tasks easy. Python can also handle scheduling (for periodic tasks like triggering a refresh, see below) and filesystem or network I/O (for logging data or retrieving assets if needed). - Use a **graphics library or framework** for drawing. If staying in Python, libraries like Pillow or Pygame could be used, but for more sophisticated visuals consider **Processing** (the Java-based environment) or **p5.js** (web canvas) if one wanted to leverage existing generative art sketches – however, those would require running a browser or the JVM,

which might be heavy on a Pi. Given the specific mention, **openFrameworks** in C++ is a strong candidate; it provides low-level control and speed, plus it has addons for image saving, which we could use to interface with the e-ink. Another alternative is **SDL2** or **Cairo** in C/C++ for drawing if openFrameworks is too large; however, openFrameworks has many conveniences (image objects, fonts, shaders, etc.) that could inspire more creative outcomes.

- **MQTT library:** If using openFrameworks, we can either integrate a C++ MQTT client (like Eclipse Paho C++ or `MQTTClient` libraries) or run a small Python MQTT subscriber that forwards data to the C++ app (via OSC messages or local socket). There's also the possibility of using **OSC (Open Sound Control)** as a protocol on localhost to send the channel values from Python to the openFrameworks app – OSC is often used in creative coding for real-time control, and it's easy to implement. This might avoid having the openFrameworks app deal with MQTT directly. But these are design choices; an all-Python or all-C++ solution can also work.
- **Home Assistant API integration patterns:** As noted, MQTT statestream is an easy way to get data out. Another pattern is using **AppDaemon** or **pyscript** within Home Assistant to forward data – but those are more for running logic inside HA. Since we want an external engine, MQTT or WebSocket streaming is the way. We should also use Home Assistant's **REST API** for any one-off data fetch if needed (e.g. to initialize the current state of all sensors at startup, since MQTT statestream might only push changes). The REST API can give the current state of any entity on demand ¹⁵ ¹⁶ . For example, on startup the engine could query the last known temperature, etc., so it doesn't start "blank". After that, the streaming updates keep it in sync.

Driving the E-Ink Display

Waveshare's 16-level grayscale e-ink display will be the canvas for our visuals. These e-paper displays have **very different characteristics** from typical screens, so careful consideration is needed in the rendering pipeline to achieve good performance and longevity:

- **Refresh Modes:** E-ink displays have multiple update modes with trade-offs between speed, ghosting, and available grayscale levels. According to documentation for similar Waveshare models, there is usually a **full refresh** (slow, ~2–5 seconds, but clears ghosting with a complete black/white flash) and a **partial refresh** mode (much faster, ~0.3–0.5 seconds for a region or the whole screen, with no full-screen blink) ¹⁷ ¹⁸ . Full refresh allows all 16 grayscale levels with optimal contrast, whereas partial refresh often works in a **bilevel (black/white)** mode or with very limited grayscale (some controllers support 1–2 additional gray levels in partial mode ¹⁹). Our strategy will be to use partial refreshes for most incremental updates (to avoid the distracting flicker) and occasionally trigger a full refresh to **reset the screen and clear any ghosting** ²⁰ ²¹ . This mirrors how e-readers operate (they only do a full flash every few page turns). One community member describes using *"partial refresh... with no flickering at all"* for quick updates, combined with a scheduled full update every N cycles to erase afterimages ²⁰ . For example, we might do quick partial updates for minor visual changes, a faster "single flicker" update every 10 minutes, and a full complete refresh once an hour ²² . The exact schedule can be tuned by observing ghosting on the specific panel, but the principle (partial most of the time, full occasionally) is well-established.
- **Partial Update Regions:** Importantly, partial mode allows updating **only a section of the display**. We should exploit this by limiting refresh to the regions that actually change. For instance, if a small

particle moves or a new shape appears in one corner, we can send just that region to the display rather than re-drawing the entire screen. This will significantly speed up the *effective* refresh and reduce wear. The Waveshare driver provides a function (or SPI commands) to set a “*partial window*” for update ²³. We can maintain a bounding box around changes each frame and only refresh that area. If our visual is mostly static with small moving elements, this means most updates involve only a fraction of the screen. However, note that many e-ink controllers require that partial refresh regions align on certain boundaries (e.g. multiples of 8 pixels) ²⁴, and they may not perfectly isolate one region from others (some ghosting or voltage bleed can occur just outside the updated area). It’s a good practice to occasionally refresh slightly beyond the minimal region or overlap regions a bit to avoid accumulating edge artifacts.

- **Avoiding Ghosting:** **Ghosting** refers to faint remnants of previous images. It’s a known issue when doing repeated partial updates without fully resetting pixels. Over time, contrast can degrade or past text leaves a slight echo. To mitigate ghosting:
- **Periodic full refresh:** as mentioned, will wipe the slate. The driver datasheet often recommends a full refresh at least every few hundred partial updates ²¹. For example, one tutorial notes: “*After several partial refreshing operations, a full refresh should be performed to remove the residual image. Otherwise the residual image may become permanent.*” ²¹. We will implement a timer or counter so that, say, every 100 partial updates or every 10 minutes (whichever sooner) triggers a full clear. This can be done during a moment where the disruption is acceptable (perhaps when house is empty or at a certain time of day, we briefly blank and refresh the screen).
- **Ghosting-aware visuals:** We can design the visuals themselves to be *ghost-tolerant*. High-contrast edges tend to leave more ghost; large areas that flip from black to white and back are noticeable. Using dithered patterns or mid-level grays can actually *mask ghosting*, as the eye is less sensitive to faint ghost in a textured gray versus a white background. In other words, a bit of *intentional noise* can make ghosting less obvious. Some community projects even use ghosting creatively – for instance, updating text in the same location with partial mode can leave a “motion blur” effect of the old text, which could be an aesthetic choice if done carefully (though we likely don’t want that here). Nonetheless, keeping most of the display in moderate gray tones and avoiding rapidly inverting large regions will naturally minimize ghosting. If a part of the image needs to frequently change from black to white, consider *cycling it through intermediate grays or patterns* instead of a hard inversion; this gives the particles in the e-ink more chance to settle and reduces abrupt ghost outlines ²⁵.
- **Manual ghost clearing:** As an intermediate between full flashes, some have used techniques like local flashing – e.g. toggling a region to black then white quickly to clear it. The Waveshare controller LUT can sometimes be customized to drive a “*clean*” for just a part of the screen. If ghosting is observed in a specific area that updates often (say an area where a shape is oscillating), the engine could occasionally force that area to blink (perhaps during the scheduled refresh interval). We will rely on the documented method (full refresh) primarily, but it’s good to know these tricks exist.
- **Partial Update Performance:** Partial refreshes on modern e-ink can be quite fast – on the order of a few hundred milliseconds for a small region ¹⁸. In practice, users have achieved ~3–4 updates per second on some black/white e-ink panels using partial mode ²⁶. Our 16-level grayscale panel might be a bit slower, but since we likely use partial in bilevel mode, we can expect maybe 0.5s or faster

updates for small regions. That's sufficient for gentle animations (2 Hz is plenty for something like a moving shape on an ambient display). We should also maximize the SPI bus speed when sending data to the panel – e.g. use the highest SPI frequency that the display's controller allows (often 10–20 MHz) and use efficient drawing routines so we spend most time waiting on the panel, not on data transfer. If using C++, we can prepare the pixel buffer and blast it via SPI quickly; if using Python, the WaveShare Python library will handle it but might be slightly slower in pure Python, so we may consider using an extension or calling C code for the SPI writes.

- **Grayscale and Dithering:** The target panel supports 16 grayscale levels (4-bit). However, in partial mode, true grayscale updates might either be unsupported or prone to artifacts ¹⁹. A known approach is to use **dithering** to approximate grays in partial updates. Dithering is the technique of mixing black and white pixels to create the illusion of intermediate shade. If we find that partial mode only does B/W, we will convert our intended grayscale output to a binary bitmap with dithering whenever doing a partial update. The engine can maintain a full grayscale image internally, but use an error-diffusion algorithm when sending it to the display. The most common is **Floyd-Steinberg dithering**, which distributes quantization error to neighboring pixels, yielding a pleasing pseudo-gray ²⁷. However, Floyd-Steinberg can produce a lot of high-frequency noise. An alternative known to work well on e-ink is **Atkinson dithering**, which diffuses a smaller portion of the error and tends to create a cleaner result (fewer “checkerboard” artifacts) ²⁸. In one e-ink art project, developers found *“Atkinson dithering works much better for low-resolution images... avoiding harsh B/W patterns and leading to a cleaner, more visually pleasing result”* ²⁸. We will likely adopt Atkinson or a similar error diffusion method when converting any grayscale buffers to 1-bit for partial updates. The Python implementation can be accelerated (the mentioned project used Numba to JIT compile the inner loops of the dithering routine ²⁹). If our panel (or a library like the **UC8151 MicroPython driver by antirez** ³⁰) allows partial refreshing with more grayscale, we could try that too. In fact, the community has seen innovations like a driver that *“displays greyscale images without relying on dithering”* and an *“anti-flicker refresh mode”* for e-ink ³¹. That particular example achieved 32 levels of gray on a UC8151-based display by carefully tuning the waveform LUT, avoiding the classic flicker (at the cost of slower updates and requiring occasional full refresh to clear ghosting) ³². It's a reminder that the e-ink controller is quite configurable – advanced users have custom-tailored waveforms to get fast partial updates and more shades (even riskily overclocking or using undocumented commands) ³³. For our prototype, we can stick to vendor-supported modes and dithering, but keep in mind that such “easter eggs” exist – e.g. one could experiment with using a medium waveform to update with 4 or 16 grays and see if ghosting is acceptable.

- **Layered Composition on E-ink:** Since we plan multiple visual layers, an important implementation detail is how to compose them for output. We have two choices: (a) flatten all layers into one final image and do a single partial update (covering the bounding box of all changes), or (b) update each layer's region separately. Option (a) is simpler and ensures we maintain correct overdraw (some e-ink controllers do not support alpha blending natively, so it must be done in software). We would maintain a full frame buffer (at 16 levels) that is the composite of all layers at the current time. Each time step, we compare this new frame to the previous frame to determine what changed. For changed regions, if small, we partial refresh them; if a large portion changed or if the composition as a whole has drifted significantly, we might refresh a larger region or the whole screen (perhaps using the fast mode with one flash if needed). The system might also intelligently separate static from dynamic content: e.g. a static background that only changes when *climate comfort* significantly shifts (maybe once an hour) can be drawn and held, and dynamic overlays (particles, etc.) drawn on

top via partial updates repeatedly. One could even avoid redrawing the static background every time by caching it in the display's memory – but in practice, it might be easier to just include it in the full buffer and rely on not touching pixels that haven't changed.

In summary, the e-ink driver subsystem will be responsible for: receiving new frame (or region) pixel data from the visualization engine, sending the proper commands for partial or full update, and managing timing (not issuing updates too fast for the panel's limits). We will use the recommended commands from Waveshare's library or the GoodDisplay controller datasheet. The engine should also handle the case of the display sleeping: e-ink panels often have a *deep sleep mode* to conserve power when idle. If our device is plugged in (e.g. a powered display on a wall), we might not need to sleep frequently – maybe only overnight. But if we do put the panel to sleep between updates, we must be cautious because, as one developer discovered, the controller's RAM loses data when powered down, which can corrupt partial updates if you try to do them immediately after waking ³⁴ ³⁵. The fix was to either avoid sleeping when using frequent partials, or to perform an initial full refresh after wake so the controller's memory matches the actual screen before attempting partial updates ³⁶. We will incorporate that knowledge: if the system has a long gap with no updates and we put the screen to sleep (to save power or lifespan), then on the next update we will do a full or fast refresh cycle to resynchronize, then resume partials.

Deployment Options: Raspberry Pi vs Mac Mini

The architecture is intended to be platform-agnostic. Both the Mac mini (with an Apple M4 or similar) and a Raspberry Pi can run the core engine. The choice may depend on where the e-ink display is attached and how much local processing we want:

- **Raspberry Pi Deployment:** The Waveshare e-ink 16-gray display typically comes as a HAT or SPI module that interfaces readily with a Raspberry Pi's GPIO pins. A Pi (e.g. a Pi 4 Model B or Pi Zero 2 W) can thus directly drive the panel using Waveshare's Python library or `spidev`. The entire stack – MQTT client, data normalization, and visual generation – can run on the Pi. We might use Python for everything on the Pi for simplicity, or compile a C++ `openFrameworks` app for the Pi if performance demands. The Pi can be small and mounted with the display in a frame, making a self-contained ambient device. One successful project, **HomePlate (Inkplate 10)**, followed this model by running an ESPHome or Pi-based app to display HA data on an e-ink screen ³⁷ ³⁸ (though HomePlate was more static dashboard). Our project, focusing on creative visuals, would similarly benefit from local execution on Pi to reduce latency and dependency on a separate machine.
- **Mac Mini Deployment:** If the Mac mini is readily available and perhaps co-located with the display (or able to communicate with it), it offers much more computing power. The Mac could run the visualization engine with heavy algorithms or even incorporate on-device AI (for instance, theoretically using a neural network to generate art – outside our scope, but feasible given the mention of AI in some contexts). The challenge is connecting the e-ink display: The Waveshare SPI HAT cannot plug directly into a Mac, but we could use a USB-to-SPI adapter or, easier, use a Raspberry Pi or microcontroller as a **bridge**. For example, the Mac could send high-level drawing commands or entire frame images to a microcontroller (ESP32 or Pi) that physically drives the e-ink. This is essentially the architecture used in that *AI e-ink art frame* project, where they hosted a “picture server” on Home Assistant/RPi and had ESP32 frames fetch images ¹³ ³⁹. In our case, we could set up a lightweight HTTP or MQTT image transport: the Mac generates a PNG image of the artwork and posts it to the Pi (which then displays it). Latency isn't a huge concern if this is done once every few

seconds at most. If using this approach, we'd implement a small receiver script on the Pi that listens for new images (over MQTT binary payload or an API call) and then uses the e-ink Python driver to refresh the screen. The Mac would then be free to use more advanced tools (even openFrameworks on macOS with Metal acceleration) to render off-screen.

- **Mixed/Relay Mode:** Another hybrid possibility: run data collection on Home Assistant (e.g. an HA automation that sends a condensed “ambient state” message periodically), and run the visualization on the Mac or Pi. However, since our engine already effectively condenses data, it's simpler to keep that functionality within the engine rather than splitting it.

In terms of development, targeting the Pi means ensuring our code is lightweight enough. openFrameworks supports Raspberry Pi (using OpenGL ES). Python with PIL on Pi can handle basic drawing at e-ink resolutions (many hobbyists use Pi + PIL for weather displays, for example). We should avoid very CPU-intensive effects or use lower resolution for them if needed. The Mac mini, conversely, has ample power to experiment with complex generative art (even 3D graphics or running a small ML model to vary the art style). We might use the Mac during development for convenience (fast iteration, better debugging tools) and then deploy to Pi for final installation.

Technical Documentation & References

To implement this system successfully, we can draw on knowledge and code from similar projects and community findings:

- **E-Ink Driver Strategies:** The Waveshare display likely uses a driver IC (such as an IT8951 or IL0373 for grayscale) for which Waveshare provides sample code. It's worth reviewing the official **Waveshare e-Paper Python library** and the controller datasheet to understand supported update modes. For partial updates, resources like the Makerguides tutorial “Partial Refresh of e-Paper Display” provide sample code and explain how to use functions like `setPartialWindow(x,y,w,h)` in popular e-ink libraries (like GxEPD for Arduino) ²³. They also demonstrate combining partial refresh with deep sleep and multiple regions, which we plan to emulate ⁴⁰ ⁴¹. The blog post “Fixing Waveshare e-paper partial updates” by Hugo*/2025 *details the discovery of the RAM volatility issue on partial refresh after delays, which guided our decision to do a full refresh after long sleeps* ⁴² ³⁴. For low-level tuning, Ben Krasnow's work on fast LUTs is illuminating: he managed to “double the refresh rate” on a 4.2” panel by modifying waveform tables, at the cost of some burn-in after thousands of cycles ⁴³ ⁴⁴. While we may not hack the LUT in our first prototype, understanding that the LUT controls ghosting vs. speed trade-off is useful if we need to fine-tune (the Hackaday article on [antirez]'s driver highlights that “controller chips are highly configurable... it's possible to do much more than the approved modes (with risk)” ³³). Another nifty idea from Kindle devices: they sometimes refresh via an “animated wipe” (line by line) to reduce perceptible flicker ⁴⁵ – while we won't implement that from scratch, it's interesting to note how the user experience can be improved by non-traditional refresh patterns.
- **Generative Art and Ambient Display Projects:** Our approach is informed by prior art in ambient visualization. The *Ambient Canvas* and *Power-Aware Cord* projects, for example, translated resource usage into artistic visuals (glowing cords, dynamic paintings) to increase awareness ⁴⁶. We aim for a similarly subtle indication. Academic research (e.g. Rodgers & Bartram's work on *ambient artistic visualization of residential resource use*) underscores the importance of “situated and timely feedback”

that is “*subtly and beautifully*” integrated ⁴⁷. They cite examples like “7000 Oaks and Counting” (which showed carbon footprint as a kaleidoscope of tree images) and “Nuage Vert” (which projected a green cloud on a power plant to reflect city energy demand) ⁴⁸ – these inspire us to think outside the box (maybe our display could metaphorically grow a tree when energy use is low, etc., in grayscale form). Another relevant reference is the concept of “Casual Infovis” and Pousman & Stasko’s criteria for ambient systems: “*display information that is important but not critical; move from the periphery to focus and back; use subtle changes to reflect updates; and be aesthetically pleasing and environmentally appropriate*” ⁴⁹. Our design aligns with this: the e-ink frame should blend into the home and only softly signal changes, never demanding immediate action.

- **Home Assistant Integration Patterns:** We will document how to set up the MQTT Statestream in Home Assistant (via `configuration.yaml`) to include the devices we want ⁸. Also, reference the Home Assistant Developer Docs for the WebSocket API as needed (the Postman quickstart and others demonstrate authenticating and subscribing to events) ⁵⁰. If using MQTT, ensure the HA MQTT broker (or external Mosquitto) is configured and that the engine’s MQTT client has the correct credentials. We might also consider using Home Assistant’s **Webhook** or **Automation** to push a summary to our engine. For instance, an HA automation could trigger whenever certain events happen and send an HTTP POST to our engine with a JSON payload of `{house_activity: 0.7, sound: 0.0, ...}`. This is a simpler but less flexible approach than streaming, and it moves the logic into HA automations. Given our desire for modularity, keeping logic in the engine (with statestream) is cleaner, but we’ll document this alternative as well.

- **Community Optimizations (“Easter Eggs”):** We will include notes on any lesser-known tricks discovered by others. For example: setting the e-ink panel’s **VCOM** voltage optimally can reduce ghosting (some Waveshare displays allow adjusting this in software or via a trimmer – too high or low VCOM can increase afterimage). Another is **thermal compensation** – e-ink refresh is temperature dependent; if the panel has a temperature sensor, we should read it and inform the controller so it can adjust the waveform (cold temperatures make e-ink sluggish and can retain ghost images longer). The community has found that **turning off** the display power or using deep sleep when idle can greatly extend life and minimize ghosting; one source notes “*after a refresh, it’s recommended to power off or hibernate the display to prolong its life*” ⁵¹ – so when the house is idle overnight, we might put the panel to sleep and wake it in the morning with a fresh screen. There’s also a fun hack: **partial refresh for animation** – some have managed to do rudimentary animations on e-ink (like the “SlowMovie” project that plays a movie at one frame per several seconds). The trick is often to update just parts of the image in sequence to give the illusion of motion without flicker. We can borrow those ideas for any subtle animations (like perhaps “wiping” the screen with a new pattern slowly from one side to the other, which could be less jarring than a full flash). If we incorporate any of these, we’ll reference the source (e.g. Ben Krasnow’s partial refresh demo video, or hackaday discussions where someone mentions “*higher-end Kindles animate refresh from left to right*” ⁴⁵).

- **Extensibility and Configuration:** We will provide documentation for the config file format and how to extend it. This includes explaining how to add a new semantic channel (e.g. “air quality”) by listing relevant HA entities (like CO2 or PM2.5 sensors) and defining how they map to a 0–1 range (perhaps via a formula or lookup table). Then, how to create a new visual layer module that subscribes to that channel value and draws something. The system could even allow enabling/disabling certain layers via config if a user finds an effect not to their taste. We’ll note that since the visuals are code, adding

an entirely new visual effect currently requires coding it as a new module, but the aim is that non-coders can still tweak mappings or simple parameters (like “particle_color: dark gray” or “background_noise_intensity: 0.5”) in a config. This separation of concerns ensures longevity: as new devices come into the home, one can integrate them by config changes alone.

In conclusion, this design balances on the line between hardware constraints and artistic expression. By **simplifying input data** into a few meaningful signals and using those to drive a thoughtfully designed generative graphic, we create a display that is *aware* of the home without being a literal dashboard. The use of e-ink lends itself to this application: its paper-like quality and slow update nature encourage calm, slow-changing visuals that can be glanced at throughout the day. With careful use of partial updates and dithering, we can achieve surprisingly dynamic effects on e-ink (as hackers have shown, even things like continuous clocks or animated images are possible ⁵²). And by building the system to be modular, we ensure it can grow – whether that means adding new sensor inputs (say a future air quality sensor channel) or swapping in a new visual theme (since an artist might want to re-skin the visualization entirely, e.g. change from abstract geometric to organic). The end result will be an **ambient home visualization frame** that runs quietly, *refreshes itself smartly to avoid burn-in* ²¹, and gives the household a gentle artistic mirror of their living environment.

References and Further Reading

- Home Assistant MQTT Statestream Integration ⁶ and WebSocket API ⁵³ – for streaming sensor updates to external applications.
- Tableau Machine ambient display research by Pousman, Romero et al. – design concepts for mapping home data to abstract art ¹ ⁵⁴.
- Waveshare e-Paper Partial Refresh techniques and ghosting mitigation: Makerguides tutorial ¹⁸ ²¹, Hugo’s blog on partial update fix ⁵⁵ ³⁴.
- Dithering algorithms for e-ink: Jimmy & Peter’s E-Ink Art Gallery guide (Atkinson vs Floyd-Steinberg) ⁵⁶ ²⁸.
- Hackaday: *Clever E-Ink Driver Does 32 Levels of Grey...* – custom LUTs for anti-flicker and direct grayscale driving ³² ⁵⁷.
- Ambient visualization examples: Rodgers & Bartram (2010) on informative art for sustainability ⁴⁷ ⁴⁶, Pousman & Stasko (2006) on Casual Infovis ⁴⁹.
- E-Ink optimization discussions: Ben Krasnow’s fast refresh experiment on 4.2” e-paper (LUT hacking, driving Teensy at 18 MHz SPI) ⁵⁸, and community notes on partial refresh limits and the need for full refresh to avoid permanent ghosting ²¹.

¹ ² ³ ⁴ ⁹ ¹⁰ ¹¹ ¹² ⁵⁴ Microsoft Word - PUC_Romero_Pousman_final2.rtf
<https://users.soe.ucsc.edu/~michaelm/publications/romero-puc-2007.pdf>

⁵ ⁴⁶ ⁴⁷ ⁴⁸ ⁴⁹ ceur-ws.org
<https://ceur-ws.org/Vol-588/102.pdf>

⁶ MQTT Statestream - Home Assistant
https://www.home-assistant.io/integrations/mqtt_statestream/

⁷ ⁵³ WebSocket API | Home Assistant Developer Docs
<https://developers.home-assistant.io/docs/api/websocket/>

- 8 Idiots guide to publish all HA devices to MQTT : r/homeassistant
https://www.reddit.com/r/homeassistant/comments/uwwn5e/idiots_guide_to_publish_all_ha_devices_to_mqtt/
- 13 14 27 28 29 39 56 Building Your Own AI & E-Ink Powered Art Gallery: A Local DIY Guide | Ticket-driven Development Blog
<https://charnley.github.io/blog/2025/04/02/e-ink-ai-esp32-local-art-gallery.html>
- 15 16 REST API - Home Assistant Developer Docs
<https://developers.home-assistant.io/docs/api/rest/>
- 17 19 20 22 34 35 36 42 55 Fixing Waveshare e-paper display partial updates
<https://thoughts.gohu.org/posts/2025/e-paper-partial-updates/>
- 18 21 23 24 40 41 51 Partial Refresh of e-Paper Display
<https://www.makerguides.com/partial-refresh-e-paper-display-esp32/>
- 25 The article mentions turning off the display to avoid ghosting and ...
<https://news.ycombinator.com/item?id=23025903>
- 26 dithernet very slow movie player - brain of mat kelcey...
http://matpalm.com/blog/dithernet_vsmp/
- 30 31 32 33 45 52 57 Clever E-Ink Driver Does 32 Levels Of Grey, Avoids Update Flicker, And More | Hackaday
<https://hackaday.com/2024/03/23/clever-e-ink-driver-does-32-levels-of-grey-avoids-update-flicker-and-more/>
- 37 HomePlate: E-Ink dashboard with Inkplate 10 - Share your Projects!
<https://community.home-assistant.io/t/homeplate-e-ink-dashboard-with-inkplate-10/383634>
- 38 Use ESPHome with e-ink Displays to blend in with your home decor!
<https://community.home-assistant.io/t/use-esphome-with-e-ink-displays-to-blend-in-with-your-home-decor/435428>
- 43 44 58 Ben Krasnow: Fast partial refresh on 4.2" E-paper display from Waveshare / Good Display
<https://benkrasnow.blogspot.com/2017/10/fast-partial-refresh-on-42-e-paper.html>
- 50 Home Assistant with WebSocket APIs - Postman Quickstarts
<https://quickstarts.postman.com/guide/home-assistant/index.html?index=.%2F..index>